Lab 1 Report- Pair 37

Kruthika Virupakshappa, Jiayi Liang

Team and ownership

Team 37: Kruthika Virupakshappa, Jiayi Liang.

Repository: github.com/KruthikaVirupakshappa/DATA266-Lab1-Pair37

Each member independently designed, trained and evaluated their own models for all three tasks, with their own architectures, hyperparameters and splits. Kruthika built a character-level GPT for Task 1, three Yelp sentiment classifiers with a mean-embedding baseline, a BiLSTM and a TextCNN for task 2, and a CycleGAN fine-tuned in two runs for Task 3. Jiayi built a character-level GPT for Task 1, three Yelp sentiment classifiers (a mean-max embedding baseline, an MLP, and a single-head self-attention model) for Task 2, and a CycleGAN trained for 10 epochs for Task 3. Shared work: the pinned dataset download scripts, the shared Task 1 and Task 2 test set and the cross-member comparison results in the Task 1 and 2 shared output folder, and the README and reproducibility records.

Task 1: GPT-style character language model

On a shared test set neither model had seen, Kruthika's 10.8M-parameter GPT predicts characters with 0.776 bits per character versus 1.335 for Jiayi's 0.45M-parameter model, while Jiayi's model generates about 11× faster.

Both members trained a decoder-only Transformer written from scratch on TinyStories, revision f54c09f, with character-level tokenization, their own char_to_idx / idx_to_char, their own 100K / 10K train / validation split, cross-entropy loss, learning-rate warm-up with cosine decay, and 10 epochs.

Approach

Aspect	Kruthika	Jiayi
Data	100K train from the official train split; 10K validation from the official validation split; train stories identical to a validation story removed (6,601 duplicates found)	100K training and 10K validation stories from the official training split, shuffled with seed 4253. The first 100K were used for training and the next 10K for validation.
Preprocessing	Mojibake repaired (e.g. “ → "), printable ASCII only; 89-character vocabulary; § end-of-story marker	Raw text; 152-character vocabulary; stories joined with newlines
Architecture	6 blocks × 6 heads, d = 384, pre-LayerNorm, GELU feed-forward (4×), learned position embeddings, context 256	2 blocks × 4 heads, d = 128, pre-LayerNorm, GELU feed-forward (4×), context 128
Training	AdamW (β 0.9/0.95, weight decay 0.1), peak LR 1e-3, 1,000 warm-up steps, cosine to 1e-4, batch 128 × 256, dropout 0.1, gradient clip 1.0, bf16	Adam, LR 3e-4, 5% warm-up, cosine to 10% of peak, batch 32 × 128
Parameters	10,814,297	452,248
Choices’ Reason	Large enough for coherent stories, still trains 10 epochs in 16 min; longer context keeps names and plot in view	This configuration was informed by prior coursework experience with a similar model that trained stably.

Team Comparison

The test set is 5,000 official-validation stories with 4.18 million characters that occur in neither member's training nor validation data. One script scores both models, each with its own preprocessing, vocabulary, and context length, after reproducing each member's own validation loss of 0.53554 and 0.92959, respectively.

Metric	**Kruthika**	**Jiayi**
Train CE (Own)	0.518	0.925289
Validation CE(Own)	0.536	0.929594
Test cross-entropy (nats)	0.537	0.924
Test perplexity	1.711	2.520
Test bits per original character	0.776	1.335
Test top-1 next-character accuracy	82.8%	70.8%
Generalization gap, own validation (val − train loss)	0.017	0.004
Distinct-1 / 2 / 3 (words, T = 0.8)	0.338 / 0.745 / 0.908	0.390 / 0.812 / 0.947
Repeated 4-gram rate (T = 0.8 / greedy)	0.7% / 3.5%	0.1% / 42.0%
Gradient norm mean / max; NaN steps	0.17 / 1.13; 0	0.85 / 2.96; 0
Training tokens/s; total training time	946,940; 15.8 min	572,170; 26.9 min

Generation chars/s (same CPU, same loop)	139	1,581
Peak GPU memory	5.3 GB	0.12 GB

Distinct-n rewards incoherence as well as variety. 1.5% of Jiayi's model generated words are not in real stories versus 0.7% of Kruthika's, this signals that Jiayi's higher values come from sentences that jump between unrelated ideas.

Failure Analysis

Kruthika's generations showed several different failure patterns. The clearest one was repetition under greedy decoding, where phrases such as "You are silly. You are not silly." repeated in a loop. This is consistent with her repeated 4-gram rate, which was much higher for greedy decoding than for sampled generation (3.5% versus 0.7%), suggesting that always choosing the most likely next character can reinforce repetitive patterns. Other failures involved consistency across the story. In one sample, an old lady suddenly says "I am a jolly boy," and a character named Sue appears without introduction. Since "old lady" had appeared only 78 characters earlier, this seems less like a context-length problem and more like the model following a strong local language pattern instead of keeping track of the character. A different example occurs when "a boy named Timmy" later "caught it with his long tail." In this case, the earlier description was 377 characters away, which is beyond the model's 256-character context window, so the model could no longer directly use that information when generating the later sentence.

Jiayi's model showed three main failure patterns. The first was repetition and broken grammar. In one sample at temperature 0.8, the model generated "One day, she saw a zipper red and wanted to share. They were very sad and was very sad and go home." which repeats the same phrase and uses inconsistent verb forms. The phrase "zipper red" also has an unnatural word order. A second sample "One day, Emily was determined to catch his mom. They both said they had a magic was stuck day long to have him a new toy car. From that day on, Lily crawled" showed problems with coherence and character consistency. It starts with Emily, then refers to "his mom," and later suddenly changes to Lily without explaining the connection. Some parts of

the sentence are also grammatically difficult to understand. At temperature 1.0, the model produced an even less coherent sample, including non-words such as “advimmering” and “trumpy.” The text still contains recognizable ideas such as the sun, fishing, and a boat, but these ideas do not connect into a clear story.

Kruthika’s error view:

Generation Setting	Generated text snippet	Failure type	Observation
Greedy (T=0) Prompt = "Tom and his mom went to the park."	"He ran to the slide. He slid down fast… "Sam, you are silly. You are not silly. You are silly. You are not silly. You are silly…"	Repetition	The story is fluent until the dialogue, then loops between two sentences until the 500-character limit. Greedy decoding always picks the most likely next character, so each repeat makes the next more likely. Repeated 4-gram rate: 8.3% greedy vs 0.19% at T=0.8.
T=0.8 Prompt = "In a big forest, there lived"	"…there lived an old lady… The old lady was very happy… They were both sad… The old lady said, "No, I'm not okay. I am a jolly boy." Sue was very sad."	Loss of coherence / character (gender) inconsistency / hallucination	The old lady calls herself "a jolly boy" even though "the old lady" is only 78 characters earlier, inside the 256-character context. A strong local pattern ("I am a … boy") overrides who the character is. "Sue" is never introduced, and "both sad" contradicts "very happy".
T=0.8 Prompt = "Once upon a time"	"Once upon a time, there was a boy named Timmy… He	Loss of coherence / contradiction with earlier context	Timmy is a boy but suddenly has a tail. "a boy named Timmy"

	threw it in the air and caught it with his long tail."		is 377 characters back, beyond the model's 256-character context window, so the model can no longer see it. Only "he / tree / climb" remain, words that often describe animals in TinyStories.

Jiayi's error view:

Generation Setting	Generated text snippet	Failure type	Observation
T=0.8 Prompt = "One day"	"One day, she saw a zipper red and wanted to share. They were very sad and was very sad and go home."	Repetition / broken grammar	"Very sad" is repeated, "zipper red" has unnatural word order, and the verb forms are inconsistent.
T=0.8 Prompt = "One day"	"One day, Emily was determined to catch his mom... From that day on, Lily crawled"	Loss of coherence / inconsistent character references / broken grammar	The story starts with Emily, then uses "his mom," and suddenly changes to Lily without explaining the relationship.
T=1.0 Prompt = "One day"	"One day, Stia came of the sun an advimmering... He smiled on a trumpy..."	Non-word generation / loss of coherence	Words such as "advimmering" and "trumpy" are not meaningful, and the ideas do not connect into a clear story.

Full failure analysis available in: Kruithka's failure analysis, Jiayi's failure analysis.

Joint analysis

Both models learned basic spelling, punctuation, and dialogue formatting, and training was stable in both cases with no NaNs. Neither model showed strong overfitting, with generalization gaps of at most 0.017. However, their weaknesses were different. The smaller model lost coherence within a sentence and was especially prone to repetition under greedy decoding, reaching a repeated 4-gram rate of 42%. The larger model generally stayed on topic better, but it still failed when important information fell outside its 256-character context window.

One limitation of our comparison is that each model was trained with only one random seed, so the results may not fully reflect run-to-run variation. As future work, we would try a mid-sized model, such as four layers with d = 256, to see where improved coherence begins to emerge without the full cost of the larger model. We would also test a longer context window, such as 512 characters, or subword tokenization so that more of each story fits within the model's effective context. Finally, top-k sampling or a repetition penalty could help reduce the looping behavior seen with greedy decoding.

Evidence	Location / File
Shared model comparison	task1_llm/shared_eval/comparison_table.md
Shared generated samples	task1_llm/shared_eval/generation_samples.txt

Member	Results	Loss Curve	Checkpoint	Raw Training Log
Kruthika	task1_llm/kruthika/results.md	task1_llm/kruthika/outputs/gpt_L6_H6_D384/loss_cu	gpt_L6_H6_D384_best.pt	task1_kruthika_gpt_L6_H6_D384_20260929-024039

		rves.png		
Jiayi	task1_llm/jiayi/results.md	task1_llm/jiayi/outputs/loss_curve_20261001_000337.png	task1_gpt_best_20261001_000337.pt	reproducibility/raw_logs/task1_jiayi_train_20261001_000337.log

Task 2: Yelp Polarity sentiment classification

Kruthika's BiLSTM is the best of the six models with a 95.3% accuracy and MCC=0.906 and significantly better than Jiayi's best model according to McNemar test, $p \approx 10^{-87}$. Jiayi's MLP slightly outperformed her baseline. Her self-attention model performed slightly worse. Kruthika's two experiments clearly beat her baseline.

Both members used the official Yelp Polarity split (fancyzhx/yelp_polarity, revision bbf1c97): a stratified 90/10 train/validation split of the 560K training reviews and the untouched official 38,000-review test set. Every model learns its own word embeddings from scratch (no pretrained embeddings or language models).

Approach

Aspect	**Kruthika**	**Jiayi**
Data analysis and cleaning	Perfectly balanced classes; no missing, duplicate or leaked reviews; literal \n sequences in 48% of reviews replaced before punctuation removal	Perfectly balanced classes; no missing or malformed rows; review-length distribution analyzed; text lowercased, negations preserved, punctuation and special characters removed, stopwords removed, and Porter stemming applied.

Preprocessing	Lowercase; "n't" → "not"; punctuation, digits and HTML entities removed; NLTK stopwords removed except 10 negations; Porter stemming; vocabulary of words seen 5+ times (40,159); max length 256	Lowercase; contractions/negations normalized (won't → will not, can't → can not, n't → not); punctuation and special characters removed while digits are retained; scikit-learn stopwords removed except no/nor/not/never; Porter stemming; 30,000-word vocabulary (min_freq = 2); reviews padded/truncated to 185 tokens
Baseline	Mean of word embeddings (d = 128) → linear; dropout 0.3; Adam LR 2e-3	Learned word embeddings (d = 128) → masked mean + max pooling → concatenate → linear → sigmoid; Adam LR 1e-3
Experiment 1	Embeddings → 1-layer BiLSTM (128 per direction) → max + mean pooling → linear; dropout 0.3; LR 1e-3	Learned word embeddings (d = 128) → masked mean pooling → MLP 128 → 64 → 32 → 1 with ReLU and dropout 0.3 → sigmoid; Adam LR 1e-3
Experiment 2	Embeddings → Conv1d with 3/4/5-word filters (128 each) → max-over-time → linear; dropout 0.5; LR 1e-3	Learned word + positional embeddings (d = 128) → single-head scaled dot-product self-attention (attention dim = 64) → masked mean pooling → linear → sigmoid; Adam LR 1e-3
Training	Batch 256, up to 5 epochs, early stopping on validation loss (patience 2), gradient clip 1.0	Batch 64; 5 epochs for all three models; Adam; BCELoss; best checkpoint selected by lowest validation loss

Why these choices	Baseline tests word choice alone. The BiLSTM captures word order and context ("not … good", "but"). The CNN captures short phrases at a fraction of the cost. Similar parameter counts isolate the architecture	Baseline tests a simple pooled embedding representation. Experiment 1 adds nonlinear classifier capacity while keeping mean-pooled embeddings. Experiment 2 adds positional information and token-to-token interactions through self-attention to test whether context-aware representations improve sentiment classification.

Team Comparison

One script recomputes every metric from both members' saved test-set probabilities after checking that both files hold the same 38,000 reviews and that all six recomputed accuracies equal each member's reported accuracy. Cross-member metrics in this table were recomputed by the shared evaluation script using a standardized 15-bin ECE. Member-local reports may use different evaluation settings.

Model	**Accuracy [95% CI]**	**Macro-F1**	**MCC**	**ROC-AUC**	**PR-AUC**	**Brier**	**ECE (15 bins)**	**Parameters**	**Train time**
K-exp1 BiLSTM	0.9531 [0.9509, 0.9550]	0.9531	0.9062	0.9902	0.9902	0.0359	0.0135	5.41M	460 s
K-exp2 TextCNN	0.9435 [0.9412, 0.9456]	0.9435	0.8871	0.9867	0.9871	0.0424	0.0049	5.34M	53 s

K-baseline mean embedding	0.9326 [0.9301, 0.9352]	0.9326	0.8652	0.9790	0.9788	0.0517	0.0118	5.14M	25 s
J-exp1 MLP	0.9283 [0.9257, 0.9309]	0.9283	0.8568	0.9796	0.9797	0.0533	0.0083	3.85M	613 s
J-baseline mean + max	0.9272 [0.9246, 0.9297]	0.9272	0.8543	0.9789	0.9789	0.0538	0.0074	3.84M	534 s
J-exp2 self-attention	0.9261 [0.9234, 0.9285]	0.9261	0.8523	0.9790	0.9794	0.0546	0.0046	3.89M	586 s

Metric	K-baseline	K-BiLSTM	K-TextCNN	J-baseline	J-MLP	J-self-attention
Precision macro / micro / weighted	0.9326 / 0.9326 / 0.9326	0.9531 / 0.9531 / 0.9531	0.9436 / 0.9435 / 0.9436	0.9272 / 0.9272 / 0.9272	0.9285 / 0.9283 / 0.9285	0.9262 / 0.9261 / 0.9262
Recall macro / micro / weighted	0.9326 / 0.9326 / 0.9326	0.9531 / 0.9531 / 0.9531	0.9435 / 0.9435 / 0.9435	0.9272 / 0.9272 / 0.9272	0.9283 / 0.9283 / 0.9283	0.9261 / 0.9261 / 0.9261
F1 micro / weighted	0.9326 / 0.9326	0.9531 / 0.9531	0.9435 / 0.9435	0.9272 / 0.9272	0.9283 / 0.9283	0.9261 / 0.9261
Macro-F1 95% CI	[0.9301, 0.9352]	[0.9509, 0.9550]	[0.9412, 0.9456]	[0.9246, 0.9297]	[0.9257, 0.9309]	[0.9234, 0.9285]

MCC 95% CI	[0.8602, 0.8703]	[0.9019, 0.9101]	[0.8825, 0.8913]	[0.8492, 0.8595]	[0.8517, 0.8619]	[0.8468, 0.8570]
Confusion matrix TN / FP / FN / TP	17,799 / 1,201 / 1,360 / 17,640	18,059 / 941 / 841 / 18,159	17,852 / 1,148 / 998 / 18,002	17,549 / 1,451 / 1,317 / 17,683	17,861 / 1,139 / 1,586 / 17,414	17,475 / 1,525 / 1,282 / 17,718

Additional slice metrics, training and inference throughput, peak memory, and the complete six-model metric breakdown are provided in the full comparison table.

McNemar test (same 38,000 reviews)	**A right, B wrong**	**A wrong, B right**	**p-value**
K-baseline vs K-BiLSTM	588	1,367	2.7×10^{-69}
K-baseline vs K-TextCNN	677	1,092	7.3×10^{-23}
J-baseline vs J-MLP	538	581	0.21
J-baseline vs J-self-attention	784	745	0.33
J-baseline vs K-baseline	627	834	7.1×10^{-8}
J-MLP (Jiayi's best) vs K-BiLSTM (Kruthika's best)	660	1,603	2.9×10^{-87}

Reviews containing negation or contrast words were more difficult for all models, with higher error rates than reviews without these features. BiLSTM achieved the lowest error rate across every evaluated slice, suggesting better robustness to negation and contrast.

Failure Analysis

Kruthika's BiLSTM error review included confident false positives and false negatives, near-threshold errors, and failures on short reviews. The mistakes came from several patterns,

especially mixed sentiment, sarcasm, off-target sentiment, and information lost during preprocessing. For example, the review “What I love about Rubios' is that they always have beer. Always. That is all I love though…” was predicted as positive with very high confidence ($p = 0.998$), even though the final sentence reverses the sentiment. A recurring issue was that preprocessing removed useful cues such as “but,” “more,” and “if,” while punctuation removal also deleted emoticons like :(, making some reviews harder to interpret correctly.

Jiayi’s MLP error review included confident false positives and false negatives, near-threshold errors, and failures on long reviews. Many mistakes came from the model relying too heavily on individual sentiment words while missing how their meaning changed in context, such as “least favorite,” “less than helpful,” sarcasm, or idiomatic expressions. Reviews with mixed sentiment, contrast words such as “but” and “however,” or later sentiment reversals were also difficult for the model. A major issue for long reviews was truncation: the model only received the first 185 processed tokens, so important positive or corrective information near the end could be lost completely. The near-threshold cases, on the other hand, were usually reviews containing genuinely conflicting positive and negative cues rather than clear high-confidence failures.

Example error view:

Error category	Kruthika example	Jiayi example
Confident False Positive	**Text**: "Very small portions, but good food. Had a perfectly cooked fillet minion, but $60 and they could have served with a toothpick… Excellent salmon, but maybe 5 bites again $60." ($p = 0.999$) **Error type**: Mixed sentiment **Testable fix:** Keep contrast words such as "but" (NLTK removes it as a stopword); retrain and compare error on the has-contrast-word slice (5.1% vs 4.1% without).	**Text:** “I have found them to be less than accomodating or helpful.” **Error type:** Comparative reversal **Testable fix:** Preserve comparative/negation terms in preprocessing.

Confident False Negative	**Text**: "This place is so much better since they changed owners… when it was the old owners, it was terrible… It was horrible. Now its much better." (p = 0.001) **Error type**: Temporal contrast **Testable fix**: Keep time markers ("now", "since", "used to"), which are currently removed as stopwords; compare errors on reviews containing them.	**Text:** “The margaritas wont do ya dirty!” **Error type:** Idiomatic expression **Testable fix:** Add bigram/trigram features so phrases like “won’t do ya dirty” can be learned as a whole.
Near-threshold Error	**Text**: "Can't get enough white chocolate" → tokens [not, get, enough, white, chocol] (p = 0.499) **Error type:** Idiomatic negation Testable fix: Add word bigrams ("get_enough") as extra tokens; measure errors on the has-negation slice (currently 4.8%).	**Text:** “The fresh fruit was good. I wouldn't pay for the meal and probably won't be back.” **Error type:** Late sentiment shift **Testable fix:** Compare with an order-sensitive model that sentiment changes near the end.
Slice-specific Failure	**Text:** "Not a lot of variety, and really nothing all that special, but what they do, they do well." → only well survives from the last clause (p = 0.002) **Error type:** Signal lost in preprocessing **Testable fix:** Keep "but" and pronoun/verb stopwords (or skip stopword removal for neural models); compare errors on the short slice (5.3%, the worst length slice).	**Text:** Early: “Cox sucks... terrible business...” Later: “why do I give this store 5 stars?... my opinion of Cox has changed.” **Error type:** Long-review truncation **Testable fix:** Increase MAX_LEN from 185 to 256.

Full failure analysis is available in Kruithka’s failure analysis, Jiayi’s failure analysis.

Joint analysis

All six models performed strongly despite learning their embeddings from scratch, with accuracy above 92.6% and low ECE <= 0.014 calibration error across the board. However, the models that ignored word order, including both baselines and Jiayi's MLP, tended to level off around 92.6% to 93.3% accuracy. This suggests that word order carries useful sentiment information, which is also reflected in the BiLSTM's larger gains on reviews involving negation and contrast. In contrast, Jiayi's single-head self-attention model did not outperform her baseline, indicating that adding one attention layer over the stemmed token sequence was not enough to provide a clear benefit in this setting.

The comparison also has some limitations because the two members used different preprocessing and training settings. For example, the vocabularies differed in size, different numbers of negation words were preserved, and the models used different learning rates, dropout settings, batch sizes, and memory strategies. Since each model was trained with only one seed, some of the observed differences may also reflect run-to-run variation. As future work, we would preserve more contrast and intensity words such as "but," "very," and "more," as well as emoticons, train the BiLSTM for more epochs since it was still improving, explore temperature scaling for calibration, and repeat experiments with several random seeds.

Member / Evidence	Metrics / Results	Training Curves	Other Evaluation Outputs	Checkpoints	Raw Logs / Manifests
Shared	task2_sentiment/shared_eval/comparison_table.md; task2_sentiment/shared_eval/mcnemar.csv	—	Six-model comparison and paired McNemar results	—	task2_shared_eval_20261005-204728.log
Kruthika	task2_sentiment/kruthika/results.md;	outputs/training_curves.png	outputs/roc_pr_calibration.png;	baseline_mean_embedding_best.pt;	task2_kruthika_sentiment_20261002-03164

	metrics_report.csv; failure_analysis.md		outputs/<model>/confusion_matrix.png; outputs/eda/class_and_length.png	exp1_bilstm_best.pt; exp2_textcnn_best.pt	3 (.log / .json)
Jiayi	task2_sentiment/jiayi/metrics_report.csv	baseline_meanmax_loss_curve_20261004_234803.png experiment1_mlp_loss_curve_20261002_211627.png experiment2_attention_loss_curve_20261003_135109.png	Confusion matrices in outputs/	task2_baseline_meanmax_best_20261004_234803.pt task2_experiment1_mlp_best_20261002_211627.pt task2_experiment2_attention_best_20261003_135109.pt	task2_jiayi_baseline_meanmax_manifest_20261004_234803.json task2_jiayi_experiment1_mlp_manifest_20261002_211627.json task2_jiayi_experiment2_attention_manifest_20261003_135109.json task2_jiayi_baseline_meanmax_train_20261004_234803.log task2_jiayi_experiment1_mlp

					_train_20261002_211627.log task2_jiayi_experiment2_attention_train_20261003_135109.log

Task 3: CycleGAN Image Style Transfer

Kruthika's CycleGAN achieved better FID/KID and stronger overall distributional performance in both directions, although Jiayi's photo-to-Monet outputs match Monet's palette more closely.

Both members trained a CycleGAN from scratch with two generators, two discriminators, adversarial and cycle-consistency losses on the class competition's 300 Monet paintings and 7,038 photos. Submissions are scored with the instructor's evaluation script: the first 300 files of each folder, FID and "MiFID" (mean paired cosine distance) per direction, and their averages over both directions. No pretrained or foundation model was used to generate or modify submitted images. Pretrained Inception-v3 and AlexNet were used only for post-hoc evaluation metrics.

Approach

Aspect	**Kruthika**	**Jiayi**
Generators	ResNet, 9 residual blocks, 64 base filters, instance norm, reflection padding; 11.4M parameters each	ResNet, 6 residual blocks, 64 base filters, instance norm, reflection padding, no dropout; 7.84M parameters each

Discriminators	70×70 PatchGAN, instance norm; 2.8M parameters each	70×70 PatchGAN (basic, 3 layers), 64 base filters, instance norm; 2.76M parameters each
Losses	LSGAN + cycle (λ 10) + identity (v1: 5, v2: 1)	LSGAN + L1 cycle-consistency loss (λA = λB = 10); no identity loss
Training aids	Image pool (50), DiffAugment (colour, translation, cutout), EMA of generator weights, bf16	Standard resize/crop/flip augmentation
Schedule	v1: 60 epochs from scratch, LR 2e-4 constant then linear decay; v2: 30-epoch fine-tune at 1e-4. Batch 4	10 epochs, Adam, LR 1e-4 constant, β = (0.5, 0.999), batch 2
Checkpoint selection	Every 2 epochs, both generators scored exactly like the evaluation script; best raw / EMA checkpoint kept	Final checkpoint after epoch 10; the same checkpoint was used for inference and evaluation
Total parameters; training time	28.3M; 7.6 h (v1 306 min + v2 150 min)	21.20M; 2415.55 s ≈ 40.3 min
Why these choices	Paper settings (Zhu et al., 2017) plus DiffAugment and EMA against the small Monet set; fine-tune because FID was still falling at epoch 60	The architecture follows the general CycleGAN design with ResNet generators, PatchGAN discriminators, Instance Normalization,

		adversarial loss, and cycle-consistency loss. A 6-block ResNet generator and trained for 10 epochs as a smaller experimental configuration. A learning rate of 0.0001 was used for conservative adversarial updates.

Comparison

Training Metrics

Metric	Kruthika	Jiayi
Generator gradient norm	27.92	39.6356
Discriminator gradient norm	14.01	11.7017
Final generator loss	2.0102	3.33775
Final discriminator loss D_A	0.0970	0.16680
Final discriminator loss D_B	0.1822	0.11045
Weighted cycle loss A	0.8591 (A + B combined)	0.97867
Weighted cycle loss B		1.18753
Identity loss	0.0802	0
Training throughput (images/s)	47.06	58.27
Peak GPU memory (MB)	7,066.4	2562.6

The table summarizes each member's CycleGAN training metrics. Because the models used different architectures and training settings, these values are reported for completeness and are not intended for direct one-to-one comparison.

To evaluate, one script scores both members' submitted prediction folders on the same 300 + 300 images, after reproducing each member's submission.csv (both within 0.05 FID; the remainder comes from different library versions).

Metric	Kruthika	Jiayi
Kaggle score (own submission)	−48.34 (Rank 17)	−54.28
Submission FID / MiFID (average of both directions)	96.27 / 0.4115	108.14 / 0.4168
Photo→Monet FID / KID	97.4 / 0.0062	108.7 / 0.0125
Photo→Monet precision / recall (k = 5)	0.55 / 0.73	0.43 / 0.74
Photo→Monet content cosine / LPIPS (input vs output)	0.81 / 0.35	0.75 / 0.52
Photo→Monet saturation / contrast (real Monets 0.305 / 0.180)	0.189 / 0.136	0.253 / 0.171
Monet→photo FID / KID	95.1 / 0.0141	107.5 / 0.0218
Monet→photo precision / recall	0.78 / 0.43	0.70 / 0.40
Monet→photo content cosine / LPIPS	0.81 / 0.34	0.79 / 0.47
Cycle-reconstruction L1 (0–1 pixel scale; own reports)	0.034 / 0.031	0.047/0.057

NaN steps	0	0

Jiayi's original cycle-reconstruction L1 reported values were 0.093/0.114 on normalized [-1,1] tensors. Values above are divided by 2 for comparison on a [0,1] scale.

With 300 images per set, even real photos compared with other real photos score FID about 81, and untouched photos versus Monets score 126.5. A perfect generator would score roughly 75–80 on average, so Kruthika's 96.3 closes about 60% of the gap between "do nothing" and "perfect".

Human audits (30 fixed samples, 2 blinded raters each)

Metric	**Kruthika**	**Jiayi**
Raters	Two friends	Two family members
Style rating (1–5)	4.1 / 5.0	2.77 / 5.0
Content preservation (1–5)	4.4 / 5.0	3.67 / 5.0
Visual artifacts	4.35	3.13
Cohen's κ / quadratic-weighted κ: style	0.877 / 0.926	0.413 / 0.882
Cohen's κ / quadratic-weighted κ: content	0.591 / 0.636	0.157 / 0.381
Cohen's κ / quadratic-weighted κ: artifacts	0.458 / 0.457	0.176 / 0.357

The two models were rated by different people, so human scores are not comparable across models; each column shows how one pair of raters judged one model. Scores were averaged across both translation directions for each criterion separately, giving overall mean ratings for style, content preservation, and artifacts. Both pairs agree most on style and least on artifacts.

Failure analysis

Kruthika's model showed several recurring visual artifacts. Photo-to-Monet outputs often had a washed-out palette, with lower saturation than real Monet paintings (0.19 vs. 0.30), and dark scenes sometimes became a grey haze. Some out-of-domain images also failed strongly, such as a Milky Way photo turning into a brown texture. Watermarks and logos could be copied into the output, while some skyscrapers remained too photographic. A dark blob also appeared repeatedly in the top-left corner of photo-to-Monet images. In Monet-to-photo translation, some hazy paintings became extremely dark, and brush strokes were sometimes still visible after translation.

Jiayi's CycleGAN showed different weaknesses in the two translation directions. Monet-to-photo often remained too painterly or blurry, while photo-to-Monet produced a stronger style transfer but sometimes lost fine details. Some outputs also changed color more than structure, suggesting that the model relied heavily on target-domain color patterns. Overall, photo-to-Monet transferred style more strongly, while Monet-to-photo preserved content better.

Joint analysis

Both CycleGANs trained stably without NaNs and were generally able to preserve the main scene content during translation. In the human evaluations, both sets of raters also preferred the photo-to-Monet direction over Monet-to-photo. However, the two models showed different trade-offs. Kruthika's model achieved the better FID score, although its outputs tended to have a more washed-out colour palette. Jiayi's model produced stronger Monet-like colour and contrast, but its FID was about 11 points worse. This suggests that palette alone does not determine FID performance, and other differences between the models, such as generator depth, training duration, and learning-rate scheduling, may also have contributed to the gap.

There are several limitations to this comparison. FID was computed using only 300 images, so the estimate may be noisy, and the two human audits used different raters. For Kruthika's model, we would test removing color augmentation from DiffAugment so that the discriminator can focus more directly on palette, reduce the cycle-consistency weight to allow stronger style changes, explore a multi-scale discriminator, and test zero padding as a possible way to reduce the corner artifact.

Member / Evidence	Results / Comparison	Training Curves / Outputs	Checkpoint	Raw Logs / Manifests
Shared	task3_gan/shared_eval/comparison_table.md	Shared evaluation outputs in task3_gan/shared_eval/	—	—
Kruthika	task3_gan/kruthika/results.md	task3_gan/kruthika/outputs/cyclegan_v1/training_curves.png; task3_gan/kruthika/outputs/cyclegan_v2_ft/training_curves.png	cyclegan_v2_ft_best.pt (epoch 18, raw weights)	task3_kruthika_cyclegan_v1_*; task3_kruthika_cyclegan_v2_ft_*
Jiayi	task3_gan/jiayi/full_metrics_report.csv; failure_analysis.md	Loss and outputs in task3_gan/jiayi/outputs/	task3_cyclegan_final_20261004_021221.pt (epoch 10) Checkpoint stored with Git LFS	task3_jiayi_cyclegan_manifest_20261004_021221.json task3_jiayi_cyclegan_train_20261004_021221.log

Reproducibility and hardware

One-command smoke test (README): python task1_llm/kruthika/src/smoke_test.py runs a short end-to-end Task 1 pipeline (11 checks, about a minute) after pip install -r requirements.txt.

Config-driven runs: every training run reads a YAML or JSON config; datasets are downloaded by pinned scripts (TinyStories f54c09f, Yelp Polarity bbf1c97, Kaggle competition data); no personal paths or credentials are committed.

Evidence trail: an unedited raw log and a manifest (package versions, config, checkpoint SHA-256) for every training and evaluation run in reproducibility/raw_logs/ and reproducibility/manifests/, including the three cross-member comparisons.

Hardware: all training ran on Google Colab GPUs; the exact GPU of every run is recorded in its raw log, manifest and metrics report. Kruthika: NVIDIA A100 (40 GB) for Tasks 1–2 and Task 3 v1, A100 (80 GB) for Task 3 v2. Jiayi: NVIDIA A100 (40 GB) for all tasks.

References

1. Vaswani, A., et al. (2017). Attention Is All You Need. *NeurIPS*.
2. Eldan, R., and Li, Y. (2023). TinyStories: How Small Can Language Models Be and Still Speak Coherent English? *arXiv:2305.07759*.
3. Zhu, J.-Y., Park, T., Isola, P., and Efros, A. A. (2017). Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks. *ICCV*.
4. Zhang, X., Zhao, J., and LeCun, Y. (2015). Character-level Convolutional Networks for Text Classification. *NeurIPS* (Yelp Polarity dataset).
5. Zhao, S., Liu, Z., Lin, J., Zhu, J.-Y., and Han, S. (2020). Differentiable Augmentation for Data-Efficient GAN Training. *NeurIPS*.
6. Heusel, M., et al. (2017). GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium (FID). *NeurIPS*.
7. Bińkowski, M., et al. (2018). Demystifying MMD GANs (KID). *ICLR*.

8. Naeem, M. F., et al. (2020). Reliable Fidelity and Diversity Metrics for Generative Models. *ICML*.
9. Zhang, R., et al. (2018). The Unreasonable Effectiveness of Deep Features as a Perceptual Metric (LPIPS). *CVPR*.
10. Chu, C., Zhmoginov, A., and Sandler, M. (2017). CycleGAN, a Master of Steganography. *NIPS Workshop*.
11. Kim, Y. (2014). Convolutional Neural Networks for Sentence Classification. *EMNLP* (TextCNN).